

Multi-Objective Optimization Using Genetic Algorithms

Mikhail Gaerlan

Computational Physics PH 4433

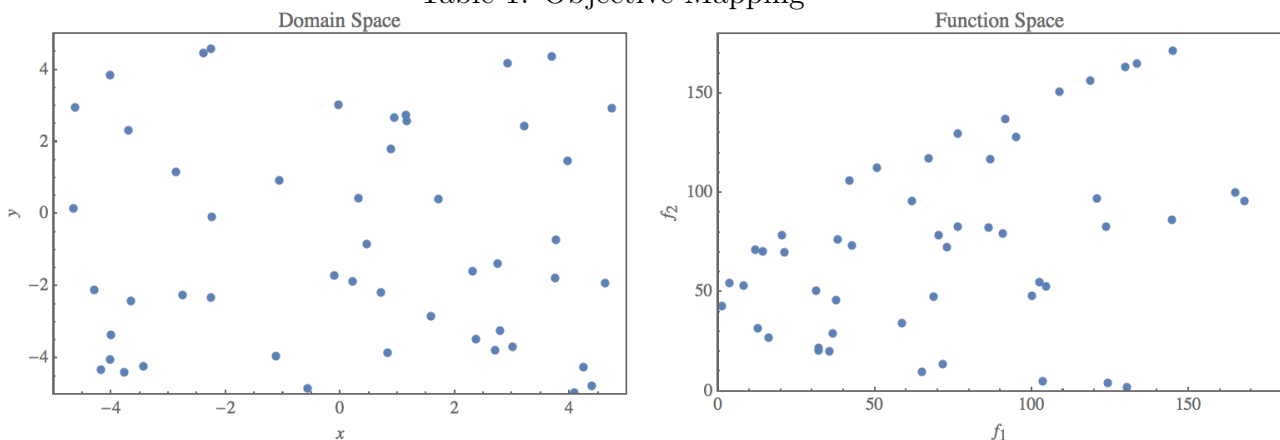
December 8, 2015

1 Optimization

Optimization is a general term for a type of numerical problem that involves minimizing or maximizing one or more functions, also called objectives. Optimization problems can come from all types of disciplines from finance to engineering and generally fall into two broad categories: single-objective or multi-objective. As the names imply, single-objective optimization deals with the optimization of a single function while multi-objective optimization deals with two or more functions simultaneously. Methods of solving optimization problems also fall under two broad categories: gradient and gradient-free methods. Gradient methods refer to the use of derivatives such as the Gauss-Newton method, and gradient free methods include genetic algorithms.

The type of method, whether gradient or gradient-free, used to solve an optimization problem depends on the function(s). Gradient methods work very well for continuous, smooth functions. The Levenberg-Marquardt algorithm is an example method for single-

Table 1: Objective Mapping



objective optimization that utilizes the Jacobian in an iterative procedure. Gradient methods, however, have a tendency to converge to local optima instead of a global optima. Genetic algorithms are examples of methods that work better for functions that are not smooth and contain multiple local minima.

Solutions of an optimization problem are vectors in some domain-space, and the objective functions maps the solutions from the domain space to some function space. For example, consider the optimization problem in Equation 1.

$$\text{Minimize } \begin{cases} f_1(x, y) = 4x^2 + 4y^2 \\ f_2(x, y) = (x - 5)^2 + (y - 5)^2 \end{cases} \quad (1)$$

The images in Table 1 shows how Equation 1 maps the thea set of solutions in the domain space $-5 \leq x, y \leq 5$ to the function space.

In a multi-objective problem, more often that not, the objectives will oppose each other; otherwise, the solution would be trivial. For example, when minimizing two objectives, a decrease in one objective would cause an increase in the other. This is known as tradeoff.

When the objectives of one solution are more optimized than all of the objectives of another solution, then the former solution dominates the latter solution. If not all the objectives of a solution are more optimized, then the solutions are non-dominating.

Instead of a single solution, there are usually sets of solutions known as fronts. Fronts are sets of solutions in which every member of the set does not dominate other members of the set. Figure 1 shows an example of a set of fronts for Equation 1. The most optimal front is known as the Pareto front, and finding the most optimal Pareto front is the goal of multi-objective optimization.

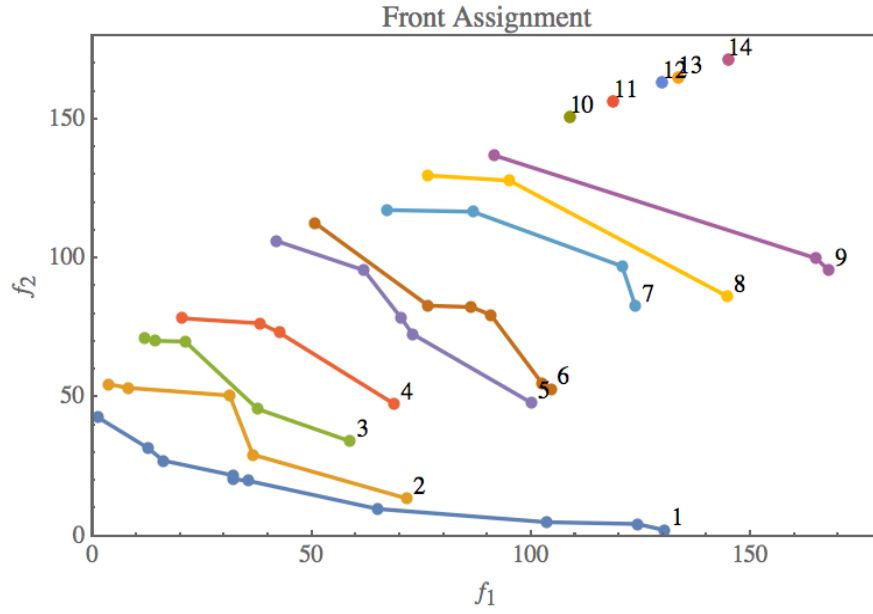


Figure 1: Front assignment.

2 Genetic Algorithms

Genetic algorithms are a class of iterative optimization methods that use the principles of evolutionary biology. When referring to genetic algorithms, some of the terminology usually associated with optimization changes to the biological terms. Solutions in a multi-objective problem are referred to as individuals, and the set of individuals is known as a population. The fitness of an individual refers to the values of the objective functions. For example in Equation 1, the individual $(1, 1)$ would have fitness values $(8, 32)$. Each iteration in the genetic algorithm is called a generation. For some generation n , the n th population is known as the offspring population while the $n - 1$ th population is the parent population.

The genetic algorithm begins with some initial population and each generation afterwards produces an offspring population using genetic operators. Genetic operators include selection, crossover, and mutation. Genetic operators use the principle of natural selection to find the best solution while using the principle of diversity to avoid convergence to a local minima.

2.1 Selection

Selection operators are used to select which offspring survive to the next generation. Truncation is the simplest selection operator which simply chooses the fittest individuals from the parent and offspring population. Tournament selection is another selection operator that randomly sorts the individuals into blocks and chooses the best individual from each block.

2.2 Crossover

Crossover operators are used to mix two or more parents to produce similar, but slightly different offspring. Most crossover operators convert the individual into binary representation to perform the operations. One-point crossover crosses the binary digits at some crossover point of two parents to create two new individuals. For example, consider the parents 7 and 18 which in binary are 00111 and 10010 respectively. Let the crossover point be position 3; thus the offspring would be 00010 and 10111 or 2 and 23 respectively. Another type of commonly used crossover operation is two-point crossover which is similar to one-point crossover but with two crossover points. Uniform crossover is a type of crossover in which each digit in the binary representation of the offspring is independently chosen from either parent with a fifty percent probability.

2.3 Mutation

Mutation operators are used to further preserve the diversity of a population to ensure nonconvergence to an optimum. A simple type of mutation involves the addition of a number chosen from some gaussian distribution, usually with a mean of 0 and a standard deviation of 1. Another type of mutation known as flip bit also performs operations on the binary representation of a number where each bit in the binary representation has some probability of being mutated.

3 Results

A variety of functions were used to test a genetic algorithm code written in Fortran. The code begins by generating a uniformly distributed population. Then, the genetic operators of selection, crossover, and mutation were applied in that order to the population for each generation. The algorithm was run until a specified amount of generations. The default genetic operators were truncation selection, two-point crossover, and gaussian mutation. The crossover probability was fifty percent while the mutation probability was twenty percent.

3.1 Operator Tests

The Binh and Korn function is described in Equation 1 and was used to test the crossover and mutation genetic operators. The population size was 100 and the algorithm was allowed to run for 10 generations. Table 2 compares the various crossover operators on the Binh and Korn function with the Gaussian mutation operator. Table 3 compares the mutation operators on the Binh and Korn function with the two-point crossover.

3.2 Convergence Test

The Ackley function as described in Equation 2 is a single-objective function used an example of genetic algorithms finding the global convergence. The population size was 50 and the algorithm was allowed to run for 5 generations.

$$f(x, y) = -20 \exp \left(-0.2 \sqrt{0.5 (x^2 + y^2)} \right) - \exp (0.5 (\cos (2\pi x) + \cos (2\pi y))) + e + 20 \quad (2)$$

Table 2: Crossover Results

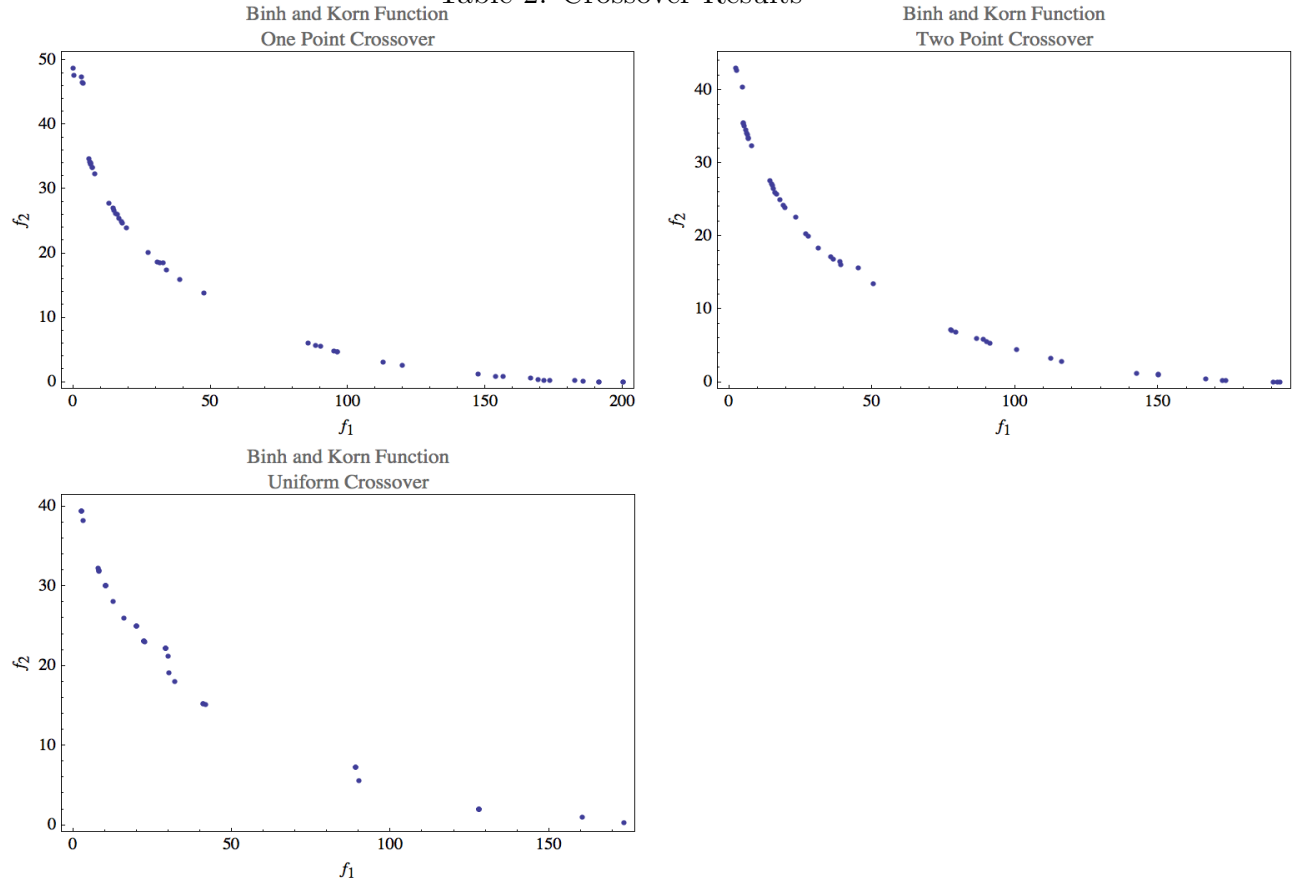


Table 3: Mutation Results

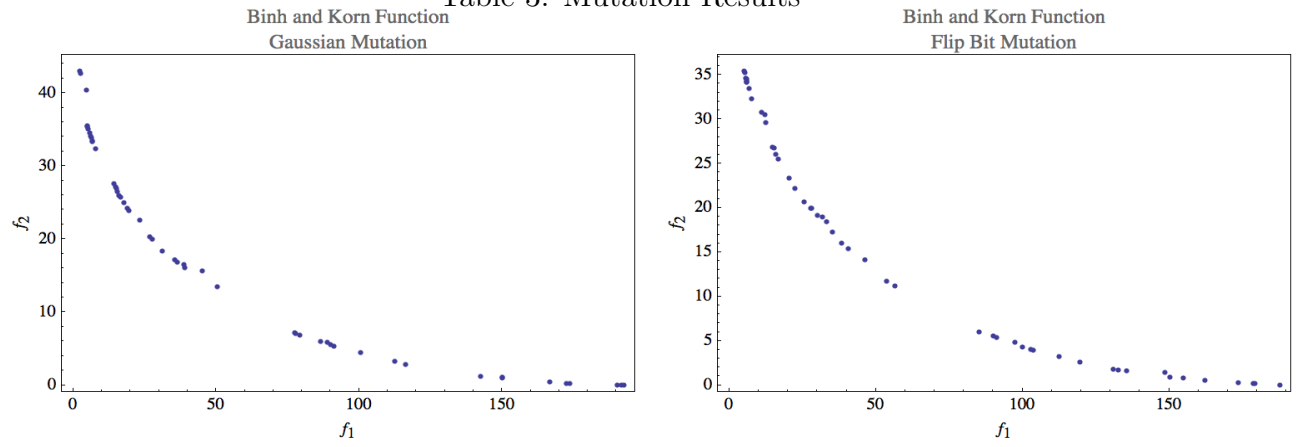


Table 4: Ackley Results

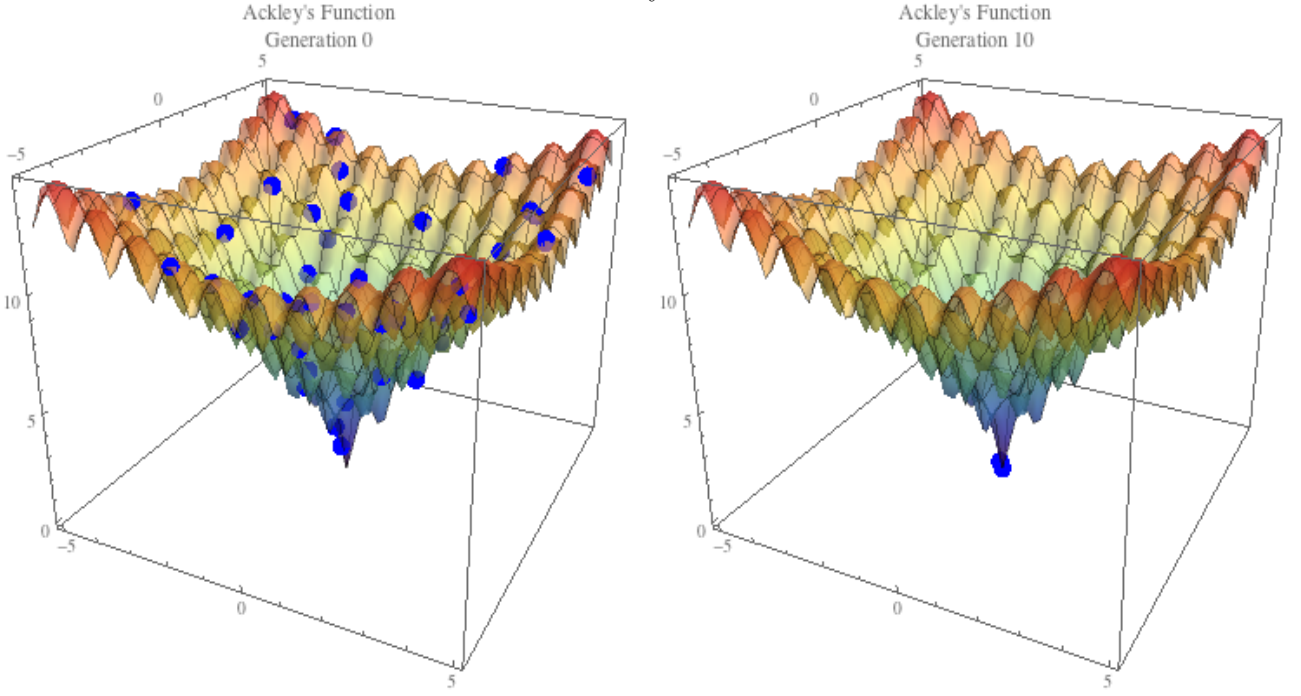


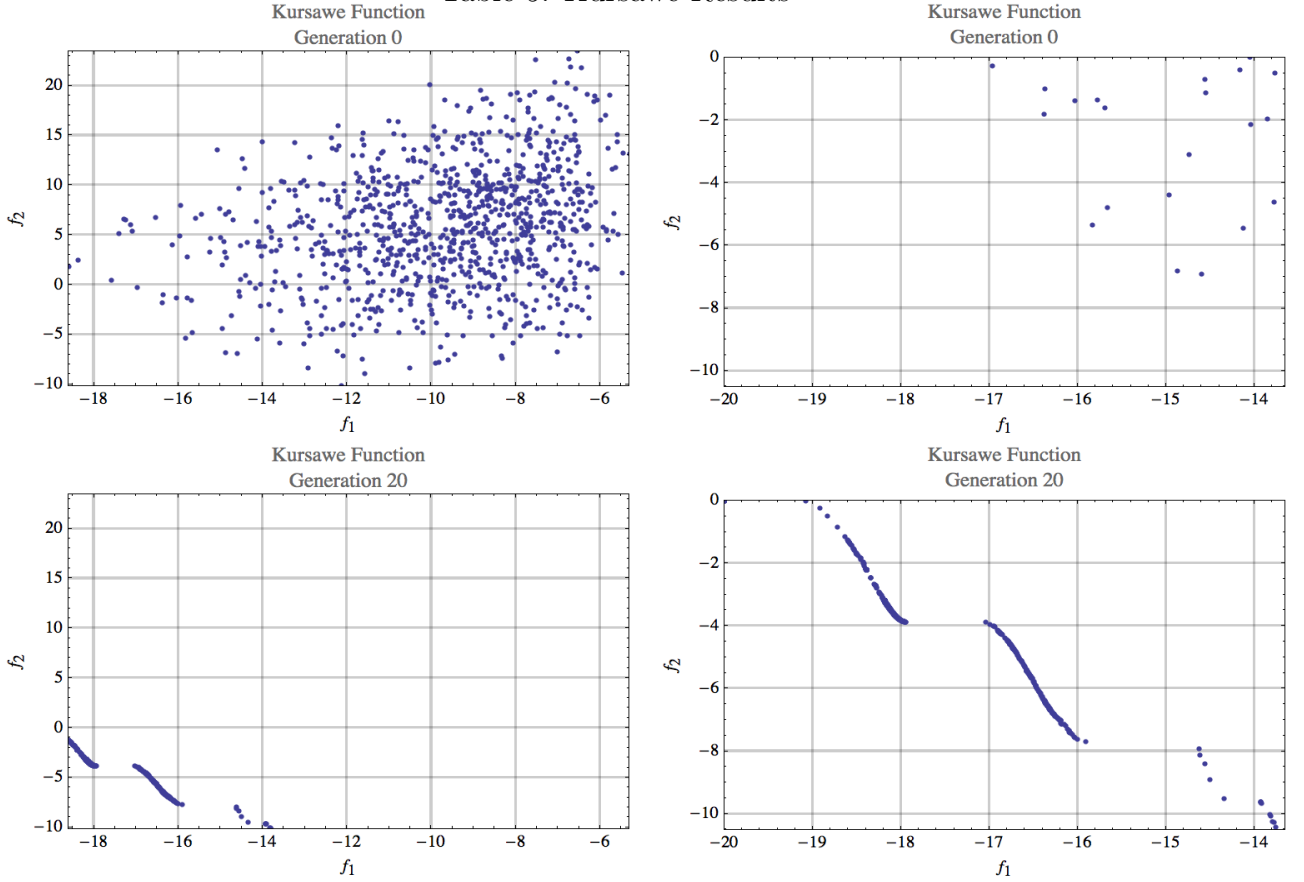
Table 4 shows the initial and final generation of solutions to the Ackley function. The function is shown by the surface while the points are the individuals in the population.

3.3 Kursawe Function

The genetic algorithm was finally used to optimize the Kursawe equation described in Equation 3. The population size was 2000 and the two-point crossover and gaussian mutation genetic operators were used. The algorithm was allowed to run for 20 generations.

$$\text{Minimize } \begin{cases} f_1(\mathbf{x}) = \sum_{i=1}^2 \left[-10 \exp \left(-0.2 \sqrt{x_i^2 + x_{i+1}^2} \right) \right] \\ f_2(\mathbf{x}) = \sum_{i=1}^3 (|x_i|^{0.8} + 5 \sin(x_i^3)) \end{cases} \quad (3)$$

Table 5: Kursawe Results



The first row of Table 5 shows the initial population while the second row shows the final population. The first column shows the full function space while the second column shows the function space zoomed into the final Pareto front of the solution.

4 Conclusion

The genetic algorithm code was successful in solving a variety of optimization problems. The genetic operators were successful in keeping diverse solutions along the Pareto front as opposed to converging to a single point along the Pareto front. The Ackley function has a clear global minimum at $(0,0)$ with multiple local minima surrounding it, but the genetic

algorithm was still able to navigate the local minima to find the global minimum. Also, unlike the solutions for the Binh and Korn function, the solutions for the Kursawe function are not immediately obvious from the initial population. However, the genetic algorithm was able to converge to a solution that is clearly better than the initial solution.

Reference:

K. Deb, *Multi-Objective Optimization using Evolutionary Algorithms* (John Wiley & Sons, 2001).